

Database Design Guidelines

The design below is a professional **logical database model** intended for **PostgreSQL + Prisma ORM**. It focuses on useful relationships without creating unnecessary abstraction.

1. Main design principles

I would use these principles throughout the database:

- **UUIDs** as primary keys.
- **createdAt** and **updatedAt** on important entities.
- **createdById** where knowing who created a record is useful.
- **Soft deletion only when historical information matters.**
- PostgreSQL **enums** for stable statuses and controlled values.
- Explicit junction tables for many-to-many relationships.
- Foreign keys to preserve referential integrity.
- Database indexes on fields used frequently for searching, filtering, or joining.
- Files stored in object storage, while PostgreSQL stores only file metadata and references. This follows the plan's storage approach.

A useful naming convention would be:

- Prisma models: **PascalCase**, such as **MemberProfile**
 - Database tables: **snake_case**, such as **member_profiles**
 - IDs: **id**
 - Foreign keys: **memberId**, **userId**, **ministryId**
 - Dates: **createdAt**, **updatedAt**, **deletedAt**
-

2. High-level database structure

CHURCH CONFIGURATION

└─ ChurchSettings

AUTHENTICATION AND ACCESS

└─ User
└─ AuthAccount
└─ UserSession
└─ Role
└─ Permission
└─ UserRole
└─ RolePermission

PEOPLE

- └─ Member
- └─ Visitor
- └─ MemberStatusHistory

MINISTRIES

- └─ Ministry
- └─ MinistryMember

ANNOUNCEMENTS

- └─ Announcement
- └─ AnnouncementDestination
- └─ AnnouncementImage

EVENTS

- └─ Event
- └─ EventParticipant

INVENTORY

- └─ InventoryCategory
- └─ StorageLocation
- └─ InventoryItem
- └─ InventoryMovement

CONTRIBUTIONS

- └─ ContributionCampaign
- └─ FinancialContribution
- └─ RequiredContributionItem
- └─ ItemCommitment

BIBLE STUDIES

- └─ BibleStudy
- └─ BibleStudyParticipant
- └─ BibleStudyMeeting
- └─ BibleStudyAttendance

DOCUMENTS

- └─ DocumentCategory
- └─ Document
- └─ DocumentAccess

CROSS-CUTTING

- └─ File
- └─ Notification
- └─ AuditLog

This results in approximately **30 core tables**, which is reasonable for the scope. The tables are separated by responsibility, but the relationships remain straightforward.

3. Church Management

ChurchSettings

Because there is only one church per deployment, this table should contain only one active record.

Field	Type	Notes
<code>id</code>	UUID	Primary key
<code>name</code>	String	Church name
<code>description</code>	Text?	Optional description
<code>logoFileId</code>	UUID?	Reference to <code>File</code>
<code>email</code>	String?	Church email
<code>phone</code>	String?	Church telephone
<code>addressLine</code>	String?	Street and number
<code>city</code>	String?	City
<code>state</code>	String?	State
<code>postalCode</code>	String?	Postal code
<code>country</code>	String	Default: Brazil
<code>website</code>	String?	Optional
<code>instagramUrl</code>	String?	Optional
<code>facebookUrl</code>	String?	Optional
<code>defaultLanguage</code>	String	Default: <code>pt-BR</code>
<code>timezone</code>	String	Example: <code>America/Boa_Vista</code>
<code>createdAt</code>	DateTime	Creation date

`updatedAt` `DateTime` Last update

Important decision

Do **not** create a `Church` table and then place `churchId` in every entity. The plan explicitly defines a separate database and deployment for each church.

To guarantee only one settings record, the application can create it during initial setup and expose it as a singleton configuration.

4. Authentication and Authorization

The authentication plan requires local credentials, Google, Apple, approval statuses, roles, permissions, provider accounts, and sessions.

User

This represents a person with access to the platform.

Field	Type	Notes
<code>id</code>	<code>UUID</code>	Primary key
<code>email</code>	<code>String</code>	Unique
<code>passwordHash</code>	<code>String?</code>	Nullable for provider-only accounts
<code>displayName</code>	<code>String</code>	Name displayed in the system
<code>status</code>	<code>UserStatus</code>	Account status
<code>emailVerifiedAt</code>	<code>DateTime?</code>	Email verification
<code>approvedAt</code>	<code>DateTime?</code>	Approval date
<code>approvedById</code>	<code>UUID?</code>	User who approved
<code>lastLoginAt</code>	<code>DateTime?</code>	Optional
<code>createdAt</code>	<code>DateTime</code>	Creation date

<code>updatedAt</code>	DateTime	Last update
<code>deletedAt</code>	DateTime?	Optional soft deletion

UserStatus

PENDING_APPROVAL
 ACTIVE
 REJECTED
 SUSPENDED
 INACTIVE

These statuses directly match the authentication plan.

AuthAccount

Stores external authentication providers.

Field	Type
<code>id</code>	UUID
<code>userId</code>	UUID
<code>provider</code>	AuthProvider
<code>providerAccountId</code>	String
<code>createdAt</code>	DateTime
<code>updatedAt</code>	DateTime

AuthProvider

GOOGLE
 APPLE

Constraint:

UNIQUE(provider, providerAccountId)

This prevents one provider account from being linked to multiple platform users.

UserSession

Field	Type
-------	------

id	UUID
userId	UUID
refreshTokenHash	String
expiresAt	DateTime
revokedAt	DateTime?
ipAddress	String?
userAgent	String?
createdAt	DateTime

Store a **hash of the refresh token**, not the raw refresh token.

RBAC tables

Role

Field	Type
id	UUID
name	String
code	String, unique
description	String?
isSystemRole	Boolean
createdAt	DateTime
updatedAt	DateTime

Examples:

SYSTEM_ADMIN
 CHURCH_ADMIN
 CHURCH_LEADER
 MINISTRY_LEADER
 MEMBER

Permission

Field	Type
id	UUID
code	String, unique
description	String?

Examples:

members.read
members.create
members.update
ministries.manage
announcements.publish
contributions.manage
documents.manage

UserRole

userId
roleId
assignedAt
assignedById

Composite primary key:

PRIMARY KEY (userId, roleId)

RolePermission

roleId
permissionId

Composite primary key:

PRIMARY KEY (roleId, permissionId)

Relationship

User
└─< UserRole >─ Role
 └─< RolePermission >─ Permission

This is flexible without being overly complicated.

5. Member Management

Member

The member should be separate from `User`.

This is important because:

- A church member may not have platform access.
- A platform user may initially be pending approval.
- A member account may be linked later.
- Member information should remain available even if the user account is deactivated.

Field	Type
<code>id</code>	UUID
<code>userId</code>	UUID?, unique
<code>firstName</code>	String
<code>lastName</code>	String
<code>birthDate</code>	Date?
<code>email</code>	String?
<code>phone</code>	String?
<code>addressLine</code>	String?
<code>city</code>	String?
<code>state</code>	String?
<code>postalCode</code>	String?
<code>membershipStatus</code>	MemberStatus
<code>membershipDate</code>	Date?
<code>notes</code>	Text?
<code>createdAt</code>	DateTime

updatedAt	DateTime
deletedAt	DateTime?

MemberStatus

ACTIVE
INACTIVE
TRANSFERRED
REMOVED

I would avoid storing an `age` field because it becomes outdated. Store `birthDate` and calculate age when needed.

MemberStatusHistory

This supports the requirement to preserve member history.

Field	Type
id	UUID
memberId	UUID
previousStatus	MemberStatus?
newStatus	MemberStatus
reason	String?
changedById	UUID
createdAt	DateTime

Relationship

User 0—1 Member
Member 1—N MemberStatusHistory

The `userId` should be nullable and unique, giving a one-to-zero-or-one relationship.

6. Ministry Management

Ministry

Field	Type
id	UUID
name	String
description	Text?
status	MinistryStatus
createdAt	DateTime
updatedAt	DateTime
createdById	UUID

MinistryStatus

ACTIVE
INACTIVE

MinistryMember

Instead of creating separate `MinistryMember` and `MinistryLeader` tables, use **one relationship table with a role field**.

Field	Type
id	UUID
ministryId	UUID
memberId	UUID
role	MinistryMemberRole
joinedAt	DateTime
leftAt	DateTime?
isActive	Boolean

createdAt DateTime

MinistryMemberRole

LEADER
MEMBER

This is simpler and supports multiple leaders naturally.

Constraint:

UNIQUE(ministryId, memberId)

Relationship:

Member
└─< MinistryMember >─ Ministry

A member may belong to multiple ministries, and a ministry may contain multiple members.

7. Announcement Management

The plan requires announcements to support multiple destinations, including all members, visitors, leaders, ministries, and all users.

Announcement

Field	Type
id	UUID
title	String
content	Text
status	AnnouncementStatus
publishedAt	DateTime?
startsAt	DateTime?

endsAt	DateTime?
createdById	UUID
eventId	UUID?
createdAt	DateTime
updatedAt	DateTime
archivedAt	DateTime?

AnnouncementStatus

DRAFT
PUBLISHED
ARCHIVED

AnnouncementDestination

Field	Type
id	UUID
announcementId	UUID
type	AnnouncementAudience
ministryId	UUID?
createdAt	DateTime

AnnouncementAudience

ALL_MEMBERS
VISITORS
CHURCH_LEADERS
MINISTRY
ALL_USERS

For a ministry destination:

type = MINISTRY
ministryId = required

For all other destinations:

ministryId = null

The backend should validate this rule.

AnnouncementImage

Field	Type
id	UUID
announcementId	UUID
fileId	UUID
displayOrder	Int

This allows multiple images without placing image URLs directly in the announcement table.

8. Event Management

Event

Field	Type
id	UUID
title	String
description	Text?
startAt	DateTime
endAt	DateTime?
location	String?
status	EventStatus

registrationEnabled	Boolean
registrationDeadline	DateTime?
maximumParticipants	Int?
createdById	UUID
createdAt	DateTime
updatedAt	DateTime

EventStatus

DRAFT
PUBLISHED
CANCELED
COMPLETED

EventParticipant

Field	Type
id	UUID
eventId	UUID
memberId	UUID
status	EventParticipationStatus
registeredAt	DateTime
checkedInAt	DateTime?

EventParticipationStatus

REGISTERED
CANCELED
ATTENDED
ABSENT

Constraint:

UNIQUE(eventId, memberId)

Relationship:

Event

└─< EventParticipant >— Member
└─< ContributionCampaign

The announcement-to-event relationship can be represented with `Announcement.eventId`. One event may have multiple announcements.

9. Inventory Management

The inventory module should distinguish between **current stock** and **stock history**.

InventoryCategory

Field	Type
<code>id</code>	UUID
<code>name</code>	String, unique
<code>description</code>	String?
<code>isActive</code>	Boolean

StorageLocation

Field	Type
<code>id</code>	UUID
<code>name</code>	String
<code>description</code>	String?
<code>isActive</code>	Boolean

InventoryItem

Field	Type
id	UUID
name	String
description	Text?
categoryId	UUID?
storageLocationId	UUID?
quantity	Decimal
minimumQuantity	Decimal
unit	String
condition	ItemCondition
isActive	Boolean
createdAt	DateTime
updatedAt	DateTime

ItemCondition

NEW
GOOD
FAIR
DAMAGED
UNUSABLE

InventoryMovement

Field	Type
id	UUID
inventoryItemId	UUID
type	InventoryMovementType

quantity	Decimal
previousQuantity	Decimal
resultingQuantity	Decimal
reason	String?
performedById	UUID
createdAt	DateTime

InventoryMovementType

ENTRY
REMOVAL
ADJUSTMENT

The current quantity remains in `InventoryItem` for fast queries, while `InventoryMovement` preserves the history.

When a movement is registered:

1. Lock or update the item in a transaction.
2. Calculate the new quantity.
3. Update `InventoryItem.quantity`.
4. Create the movement record.
5. Commit the transaction.

10. Contribution Management

The plan has two contribution models: financial and item-based.

A clean design is to use **one common campaign table** and separate tables for each contribution type.

ContributionCampaign

Field	Type
id	UUID

<code>title</code>	String
<code>description</code>	Text?
<code>type</code>	ContributionType
<code>status</code>	ContributionCampaignStatus
<code>eventId</code>	UUID?
<code>startsAt</code>	DateTime?
<code>endsAt</code>	DateTime?
<code>financialTarget</code>	Decimal?
<code>createdById</code>	UUID
<code>createdAt</code>	DateTime
<code>updatedAt</code>	DateTime
<code>closedAt</code>	DateTime?

ContributionType

FINANCIAL
ITEM

ContributionCampaignStatus

DRAFT
ACTIVE
CLOSED
ARCHIVED

For a financial campaign:

`type` = FINANCIAL
`financialTarget` = optional or required

For an item campaign:

`type` = ITEM
`financialTarget` = null

FinancialContribution

Field	Type
id	UUID
campaignId	UUID
memberId	UUID?
amount	Decimal(12,2)
contributionCreatedAt	DateTime
paymentMethod	PaymentMethod?
reference	String?
recordedById	UUID
createdAt	DateTime

PaymentMethod

CASH
PIX
BANK_TRANSFER
CARD
OTHER

`memberId` may be nullable if anonymous contributions are allowed.

The total collected should generally be calculated with:

SUM(financial_contributions.amount)

Avoid storing `amountCollected` in `ContributionCampaign` unless performance later proves it necessary. Calculating it avoids synchronization problems.

RequiredContributionItem

Field	Type
id	UUID
campaignId	UUID

name	String
description	String?
requiredQuantity	Decimal
maximumContributors	Int?
unit	String
createdAt	DateTime
updatedAt	DateTime

ItemCommitment

Field	Type
id	UUID
requiredItemId	UUID
memberId	UUID
quantity	Decimal
status	ItemCommitmentStatus
committedAt	DateTime
approvedAt	DateTime?
approvedById	UUID?
canceledAt	DateTime?
canceledById	UUID?
notes	String?

ItemCommitmentStatus

PENDING

APPROVED
CANCELED
FULFILLED

The system should calculate:

committed quantity
= SUM(active commitments)

remaining quantity
= required quantity - committed quantity

number of contributors
= COUNT(DISTINCT memberId)

The limits must be validated inside a database transaction because simultaneous commitments could otherwise exceed the allowed quantity.

11. Bible Studies Management

The original plan mentions schedules and attendance. To keep this flexible without overdesigning, I recommend separating the study from its meetings.

BibleStudy

Field	Type
id	UUID
title	String
topic	String?
description	Text?
status	BibleStudyStatus
ministryId	UUID?
createdById	UUID

`createdAt` `DateTime`

`updatedAt` `DateTime`

BibleStudyStatus

PLANNED

ACTIVE

COMPLETED

CANCELED

BibleStudyParticipant

Field	Type
<code>id</code>	UUID
<code>bibleStudyId</code>	UUID
<code>memberId</code>	UUID
<code>role</code>	BibleStudyRole
<code>joinedAt</code>	DateTime
<code>isActive</code>	Boolean

BibleStudyRole

LEADER

PARTICIPANT

Constraint:

UNIQUE(`bibleStudyId`, `memberId`)

This removes the need for a separate `BibleStudyLeader` table.

BibleStudyMeeting

Field	Type
<code>id</code>	UUID
<code>bibleStudyId</code>	UUID

<code>scheduledAt</code>	DateTime
<code>endsAt</code>	DateTime?
<code>location</code>	String?
<code>topic</code>	String?
<code>notes</code>	Text?
<code>status</code>	MeetingStatuses

MeetingStatus

SCHEDULED
COMPLETED
CANCELED

BibleStudyAttendance

Field	Type
<code>id</code>	UUID
<code>meetingId</code>	UUID
<code>memberId</code>	UUID
<code>status</code>	AttendanceStatuses
<code>recordedAt</code>	DateTime
<code>recordedById</code>	UUID

AttendanceStatus

PRESENT
ABSENT
EXCUSED

Constraint:

UNIQUE(meetingId, memberId)

This design makes attendance accurate because attendance belongs to a **specific meeting**, not only to the overall Bible study.

12. Document Management

The plan requires documents to be categorized, associated with ministries, events, and Bible studies, and protected according to access permissions.

File

This is the central metadata table for uploaded files.

Field	Type
id	UUID
storageKey	String, unique
originalName	String
mimeType	String
sizeBytes	BigInt
uploadedById	UUID
createdAt	DateTime

Do not store the full file binary in PostgreSQL.

DocumentCategory

Field	Type
id	UUID
name	String, unique
description	String?

isActive Boolean

Document

Field	Type
id	UUID
title	String
description	Text?
categoryId	UUID?
fileId	UUID
status	DocumentStatus
ministryId	UUID?
eventId	UUID?
bibleStudyId	UUID?
uploadedById	UUID
createdAt	DateTime
updatedAt	DateTime
archivedAt	DateTime?

DocumentStatus

ACTIVE
ARCHIVED
DELETED

A document can be linked to:

- no specific entity;
- one ministry;
- one event;
- one Bible study.

This is simple and matches the current requirements.

If future requirements allow one document to be associated with many events or many studies, junction tables can be added later. I would **not add them now** because they would make the initial structure more complex without a current need.

DocumentAccess

Field	Type
id	UUID
document Id	UUID
accessType	DocumentAccessType
roleId	UUID?
ministry Id	UUID?

DocumentAccessType

ALL_AUTHENTICATED_USERS
ROLE
MINISTRY

Examples:

Document A
→ ALL_AUTHENTICATED_USERS

Document B
→ ROLE: CHURCH_ADMIN

Document C
→ MINISTRY: PRAISE_MINISTRY

The backend evaluates these rules before returning a document or generating a download URL.

13. Visitor Management

Visitors appear as an announcement destination, but a full visitor module is not currently included in the main implementation order.

For the initial version, keep this entity small:

Visitor

Field	Type
<code>id</code>	UUID
<code>fullName</code>	String
<code>email</code>	String?
<code>phone</code>	String?
<code>firstVisitAt</code>	DateTime
<code>status</code>	VisitorStatus
<code>notes</code>	Text?
<code>createdAt</code>	DateTime
<code>updatedAt</code>	DateTime

VisitorStatus

ACTIVE
FOLLOWED_UP
BECAME_MEMBER
INACTIVE

This supports announcement targeting without implementing a complete visitor follow-up module now.

14. Notifications

The plan includes in-app notifications as an initial feature and additional delivery channels later.

Notification

Field	Type
id	UUID
userId	UUID
type	String
title	String
message	Text
resourceType	String?
resourceId	UUID?
readAt	DateTime?
createdAt	DateTime

Examples:

type = ANNOUNCEMENT_PUBLISHED
resourceType = ANNOUNCEMENT
resourceId = announcement UUID

I would keep this generic because notifications are a cross-module concern.

15. Audit Logs

The plan requires important administrative actions to be recorded.

AuditLog

Field	Type
id	UUID

<code>userId</code>	UUID?
<code>action</code>	String
<code>entityType</code>	String
<code>entityId</code>	UUID?
<code>previousData</code>	JSONB?
<code>newData</code>	JSONB?
<code>ipAddress</code>	String?
<code>createdAt</code>	DateTime

Examples:

```

action = MEMBER_CREATED
entityType = MEMBER
entityId = member UUID

```

or:

```

action = ROLE_ASSIGNED
entityType = USER_ROLE
entityId = user-role UUID

```

I would use **JSONB** only here because audit data is naturally variable. Avoid using JSONB as a replacement for properly modeled relational data.

16. Main relationship diagram

USER

```

├── 0..1 MEMBER
├── N AUTH_ACCOUNTS
├── N USER_SESSIONS
├── N USER_ROLES
├── N AUDIT_LOGS
└── N NOTIFICATIONS

```

ROLE

```

└── N ROLE_PERMISSIONS — PERMISSION

```

MEMBER

- ├── N MINISTRY_MEMBERS — MINISTRY
- ├── N EVENT_PARTICIPANTS — EVENT
- ├── N FINANCIAL_CONTRIBUTIONS
- ├── N ITEM_COMMITMENTS
- ├── N BIBLE_STUDY_PARTICIPANTS — BIBLE_STUDY
- └── N BIBLE_STUDY_ATTENDANCE

ANNOUNCEMENT

- ├── N ANNOUNCEMENT_DESTINATIONS
- ├── N ANNOUNCEMENT_IMAGES — FILE
- └── 0..1 EVENT

EVENT

- ├── N EVENT_PARTICIPANTS
- └── N CONTRIBUTION_CAMPAIGNS

CONTRIBUTION_CAMPAIGN

- ├── N FINANCIAL_CONTRIBUTIONS
- └── N REQUIRED_CONTRIBUTION_ITEMS
 - └── N ITEM_COMMITMENTS

INVENTORY_ITEM

- ├── 1 INVENTORY_CATEGORY
- ├── 1 STORAGE_LOCATION
- └── N INVENTORY_MOVEMENTS

BIBLE_STUDY

- ├── N BIBLE_STUDY_PARTICIPANTS
- ├── N BIBLE_STUDY_MEETINGS
 - └── N BIBLE_STUDY_ATTENDANCE
- └── N DOCUMENTS

DOCUMENT

- ├── 1 FILE
 - ├── 0..1 MINISTRY
 - ├── 0..1 EVENT
 - ├── 0..1 BIBLE_STUDY
 - └── N DOCUMENT_ACCESS
-

17. Important database constraints

These should be implemented through PostgreSQL constraints, Prisma constraints where supported, and backend validation.

Unique constraints

User.email

AuthAccount(provider, providerAccountId)

Member.userId

Role.code

Permission.code

Ministry.name

InventoryCategory.name

DocumentCategory.name

MinistryMember(ministryId, memberId)

EventParticipant(eventId, memberId)

BibleStudyParticipant(bibleStudyId, memberId)

BibleStudyAttendance(meetingId, memberId)

Check constraints

InventoryItem.quantity >= 0

InventoryItem.minimumQuantity >= 0

FinancialContribution.amount > 0

RequiredContributionItem.requiredQuantity > 0

ItemCommitment.quantity > 0

Event.endAt > Event.startAt

ContributionCampaign.endsAt > ContributionCampaign.startsAt

Referential behavior

Recommended defaults:

- **RESTRICT** for important historical records.
- **SET NULL** for optional creator or approver references.
- **CASCADE** for pure junction records.

Examples:

Delete Ministry

→ Restrict if members or documents are still associated

Delete Announcement

→ Cascade AnnouncementDestination

→ Cascade AnnouncementImage

Delete Event

→ Restrict if participants or campaigns exist

Delete User

→ Do not physically delete

→ Use status or deletedAt

18. Recommended indexes

Do not index every field. Index fields that are frequently searched, filtered, joined, or sorted.

Users

User(email)

User(status)

Members

Member(lastName, firstName)

Member(membershipStatus)

Member(email)

Member(phone)

For more advanced name searching later, PostgreSQL [pg_trgm](#) can be introduced.

Ministries

Ministry(status)

MinistryMember(memberId)

MinistryMember(ministryId, isActive)

Announcements

Announcement(status, publishedAt)

Announcement(startsAt, endsAt)

AnnouncementDestination(ministryId)

Events

Event(status, startAt)

EventParticipant(memberId)

Inventory

InventoryItem(categoryId)

InventoryItem(storageLocationId)
InventoryItem(name)
InventoryMovement(inventoryItemId, createdAt)

Contributions

ContributionCampaign(type, status)
ContributionCampaign(eventId)
FinancialContribution(campaignId, contributedAt)
ItemCommitment(requiredItemId, status)
ItemCommitment(memberId)

Bible studies

BibleStudy(status)
BibleStudyParticipant(memberId)
BibleStudyMeeting(bibleStudyId, scheduledAt)

Documents

Document(categoryId)
Document(status)
Document(ministryId)
Document(eventId)
Document(bibleStudyId)

19. Important design decisions

Keep **User** and **Member** separate

This is one of the most important decisions.

Do not put all member information directly in **User**, because the concepts are different:

User = authentication and platform access

Member = church membership information

The relationship is:

User 0—1 Member

A member can exist without a user account.

Use one ministry relationship table

Do not create:

MinistryMember

MinistryLeader

Instead:

MinistryMember

- |— ministryId
- |— memberId
- |— role = LEADER | MEMBER

This reduces duplication and supports multiple leaders.

Use one Bible-study participant table

Do not create:

BibleStudyLeader

BibleStudyParticipant

Instead:

BibleStudyParticipant

- |— bibleStudyId
- |— memberId
- |— role = LEADER | PARTICIPANT

Keep contribution campaigns unified

Do not create completely independent campaign entities unless future business rules become substantially different.

Use:

ContributionCampaign

- |— type = FINANCIAL
 - |— FinancialContribution
- |— type = ITEM
 - |— RequiredContributionItem
 - |— ItemCommitment

This gives common campaign behavior while preserving different contribution data.

Do not store calculated values unnecessarily

Avoid storing:

Member.age
Campaign.totalCollected
RequiredItem.quantityCommitted
RequiredItem.quantityRemaining

Calculate them from source data.

This prevents inconsistent values.

For example:

quantityRemaining
=
requiredQuantity
-
SUM(active commitments)

If performance later requires caching, introduce it deliberately with transaction-based updates.

20. Recommended implementation order

I would implement the database in this order:

Phase 1 — Foundation

1. ChurchSettings
2. User
3. AuthAccount
4. UserSession
5. Role
6. Permission
7. UserRole
8. RolePermission
9. File

10. AuditLog

Phase 2 — Core church structure

11. Member

12. MemberStatusHistory

13. Ministry

14. MinistryMember

Phase 3 — Communication and activities

15. Visitor

16. Event

17. EventParticipant

18. Announcement

19. AnnouncementDestination

20. AnnouncementImage

Phase 4 — Operational modules

21. InventoryCategory

22. StorageLocation

23. InventoryItem

24. InventoryMovement

25. ContributionCampaign

26. FinancialContribution

27. RequiredContributionItem

28. ItemCommitment

Phase 5 — Learning and documents

29. BibleStudy

30. BibleStudyParticipant

31. BibleStudyMeeting

32. BibleStudyAttendance

33. DocumentCategory

34. Document

35. DocumentAccess

Phase 6 — Cross-module functionality

36. Notification